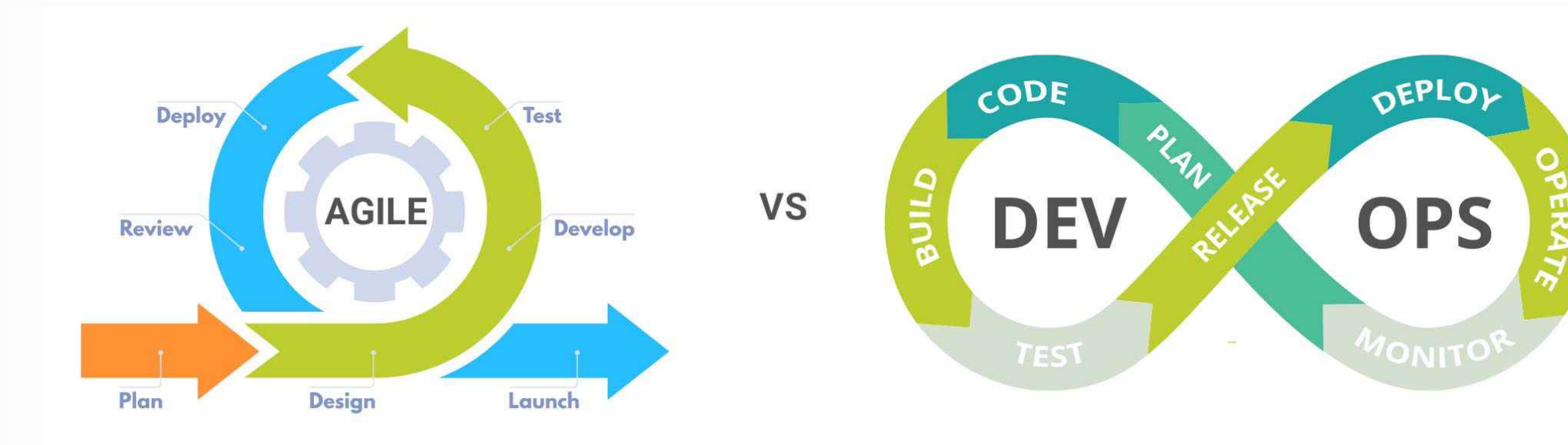


Ingegneria, Gestione ed Evoluzione del Software

DevOps

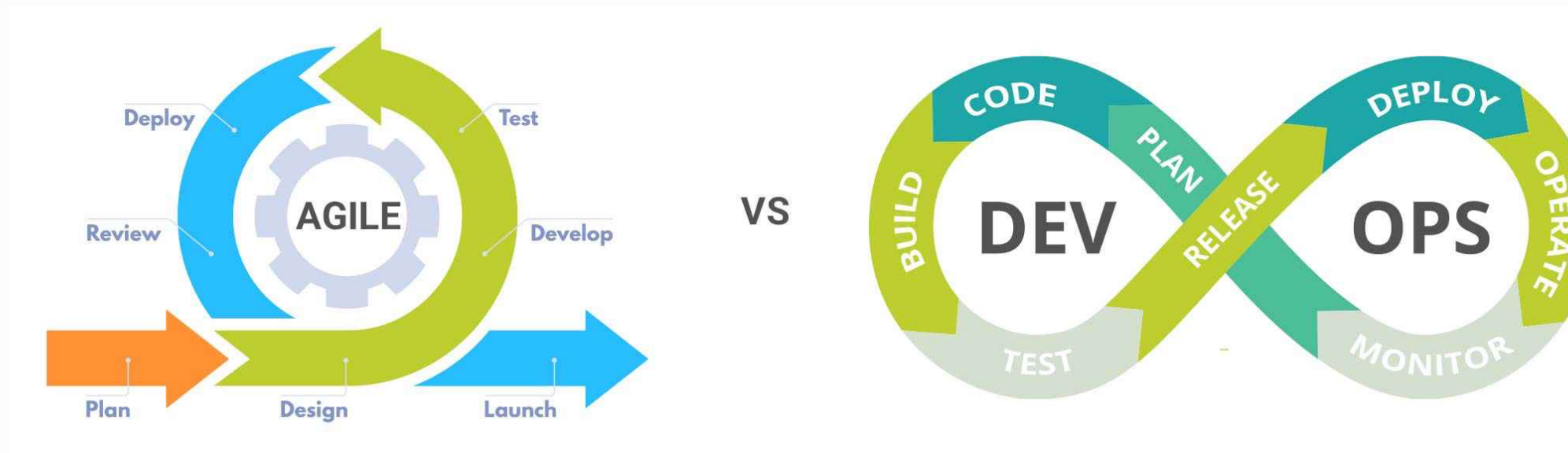


Un po' di contesto su DevOps



DevOps è il risultato **dell'applicazione dei principi più affidabili** provenienti dal campo della **produzione fisica e della leadership al flusso di valore IT**.

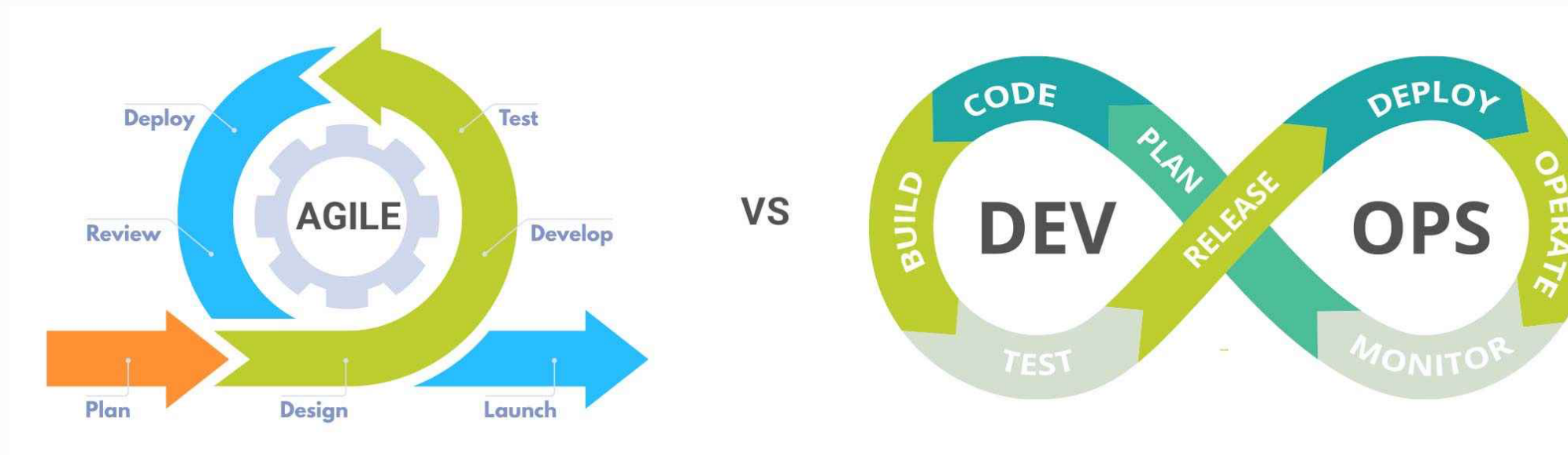
Un po' di contesto su DevOps



DevOps è il risultato **dell'applicazione dei principi più affidabili** provenienti dal campo della **produzione fisica e della leadership al flusso di valore IT**.

DevOps si basa su **body of knowledge** provenienti da **Lean, Theory of Constraints, il Sistema di Produzione Toyota**, ingegneria della resilienza, organizzazioni di apprendimento, cultura della sicurezza, fattori umani e molti altri.

Un po' di contesto su DevOps

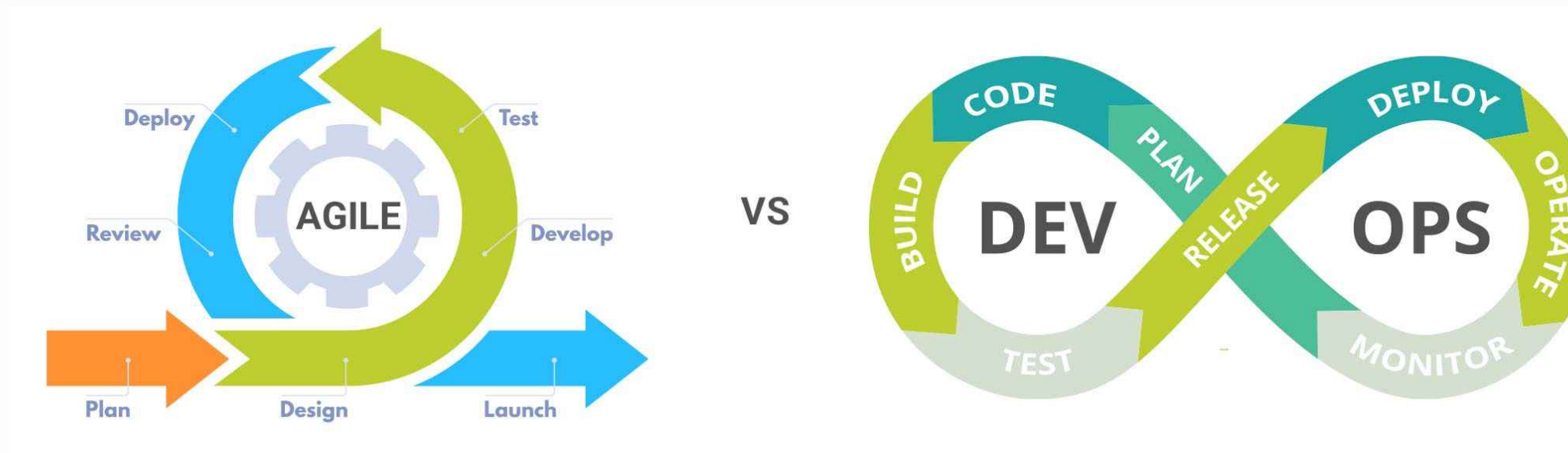


DevOps è il risultato **dell'applicazione dei principi più affidabili** provenienti dal campo della **produzione fisica e della leadership al flusso di valore IT**.

DevOps si basa su **body of knowledge** provenienti da **Lean, Theory of Constraints, il Sistema di Produzione Toyota**, ingegneria della resilienza, organizzazioni di apprendimento, cultura della sicurezza, fattori umani e molti altri.

Il principio chiave è "**consegnare frequentemente software funzionante, da un paio di settimane a un paio di mesi, con una preferenza per il periodo più breve**", sottolineando il desiderio di dimensioni di lotto ridotte, rilasci incrementali invece di rilasci grandi secondo l'approccio waterfall.

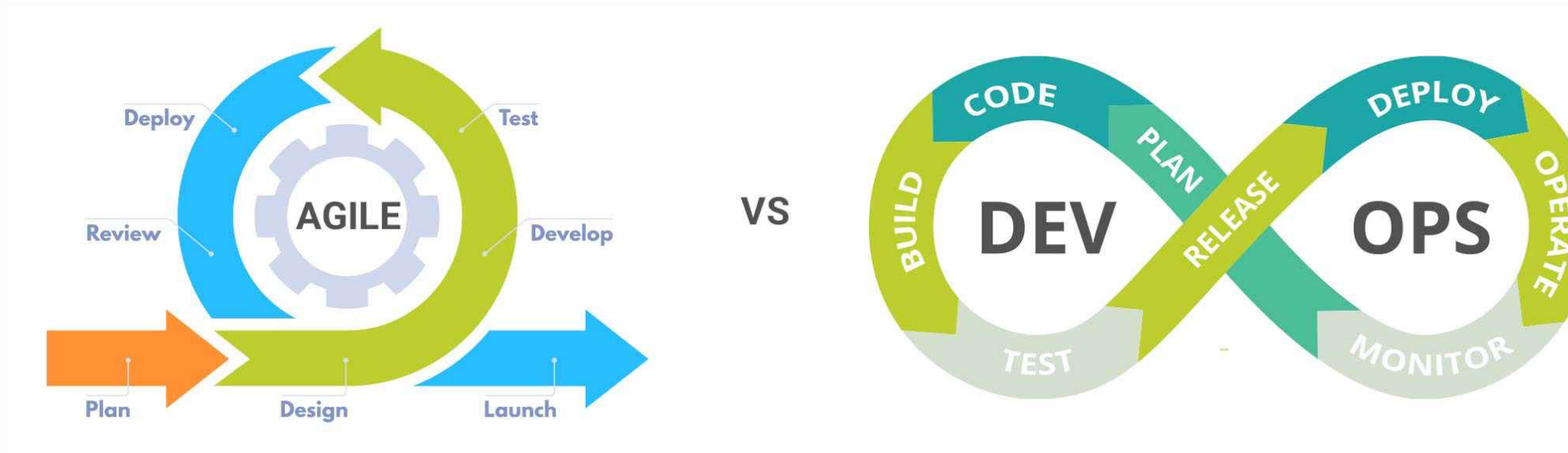
Un po' di contesto su DevOps



Value Stream (Flusso di valore): “La sequenza di attività necessarie per **progettare, produrre e consegnare** un **bene o un servizio** a un cliente, inclusi i flussi duali di informazioni e materiali.”

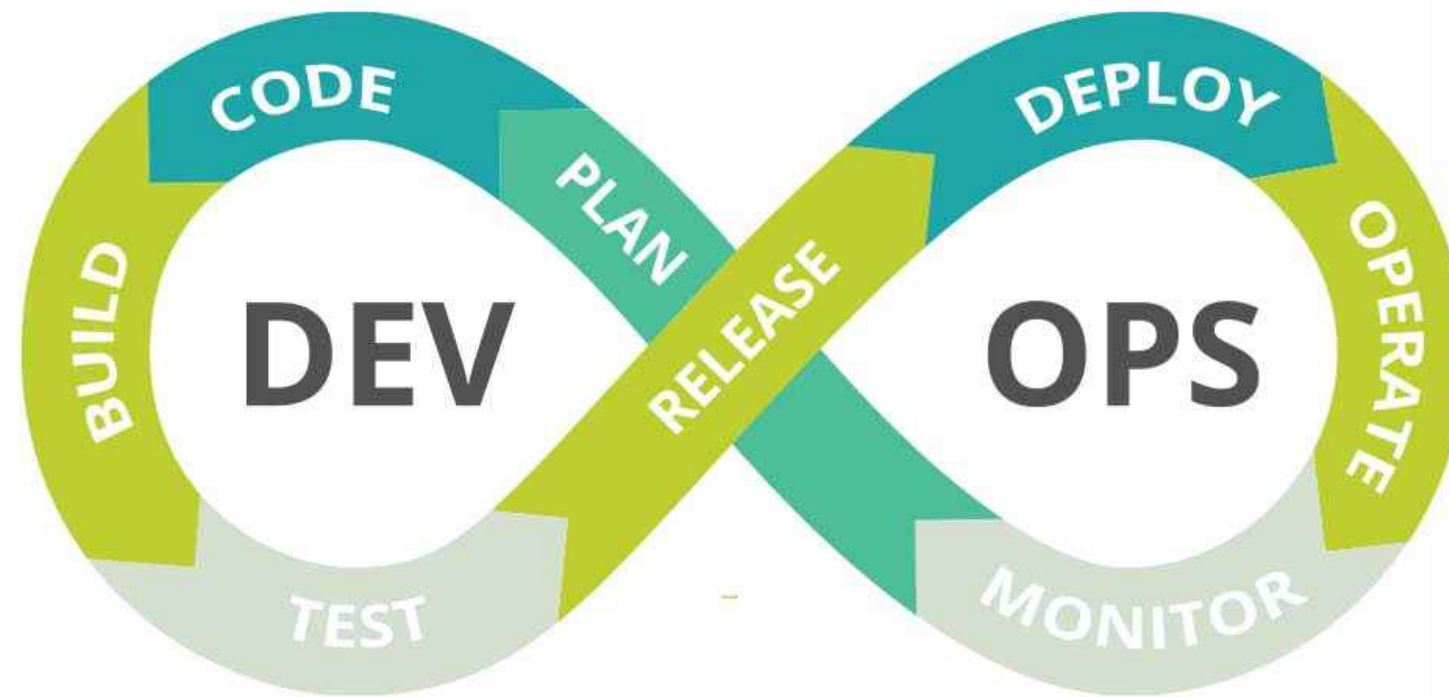
In DevOps, di solito definiamo il nostro flusso di valore tecnologico come il processo necessario per trasformare un'ipotesi aziendale in un servizio abilitato dalla tecnologia che offre valore al cliente.

Un po' di contesto su DevOps



Value Stream (Flusso di valore): “La sequenza di attività necessarie per **progettare, produrre e consegnare** un **bene o un servizio** a un cliente, inclusi i flussi duali di informazioni e materiali.”

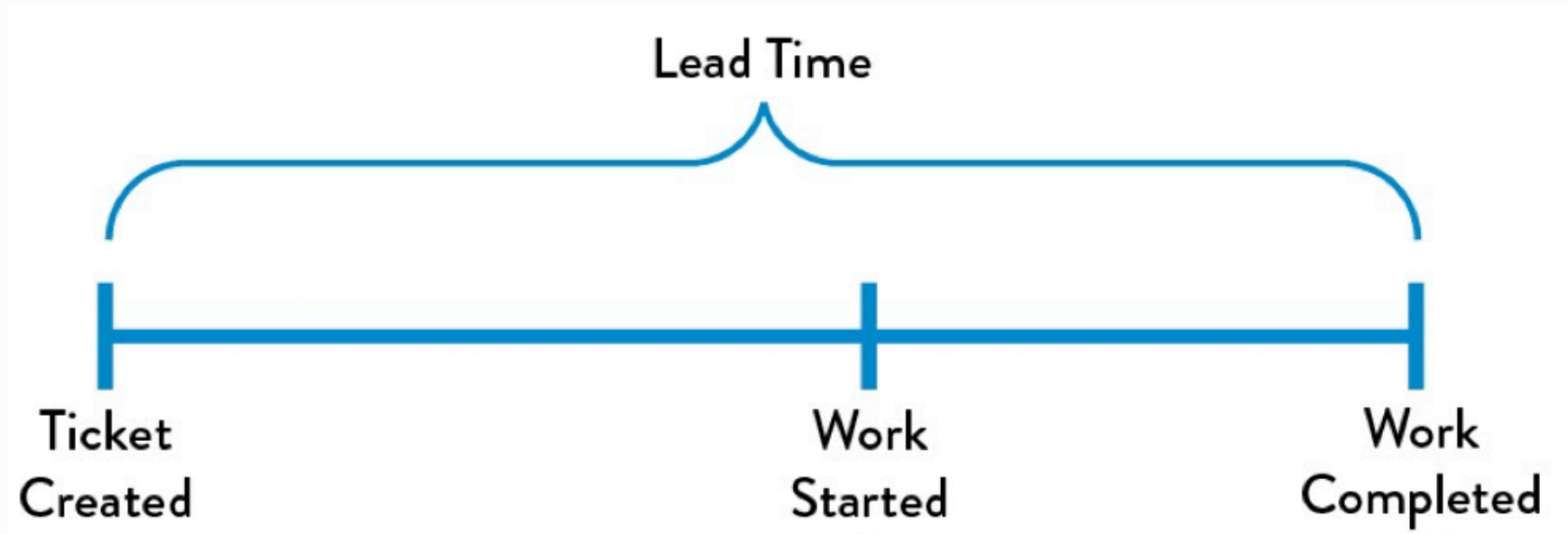
In DevOps, di solito definiamo il nostro flusso di valore tecnologico come il **processo necessario per trasformare un'ipotesi aziendale** in un **servizio abilitato dalla tecnologia** che offre valore al cliente.



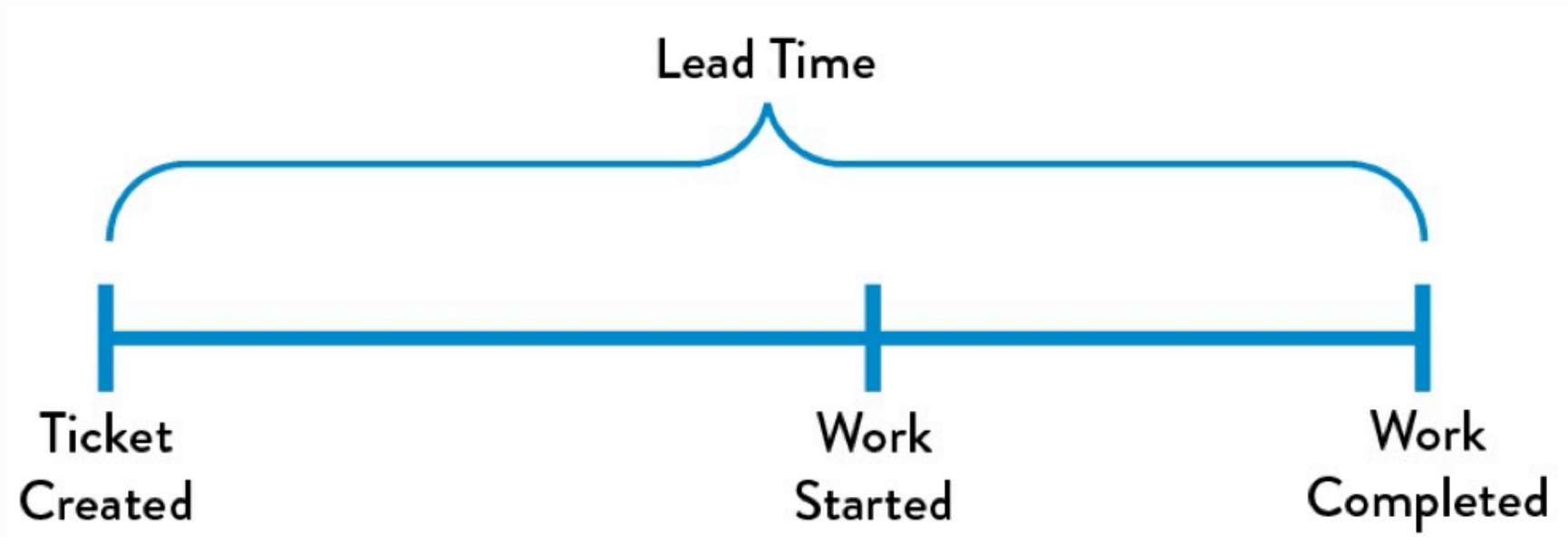
DevOps punta a **minimizzare il value stream.**

A questo punto definiamo:

“DevOps è un insieme di pratiche volte a ridurre il tempo che intercorre tra il commit di una modifica a un sistema e la modifica nella produzione normale, garantendo al contempo un'elevata qualità.”

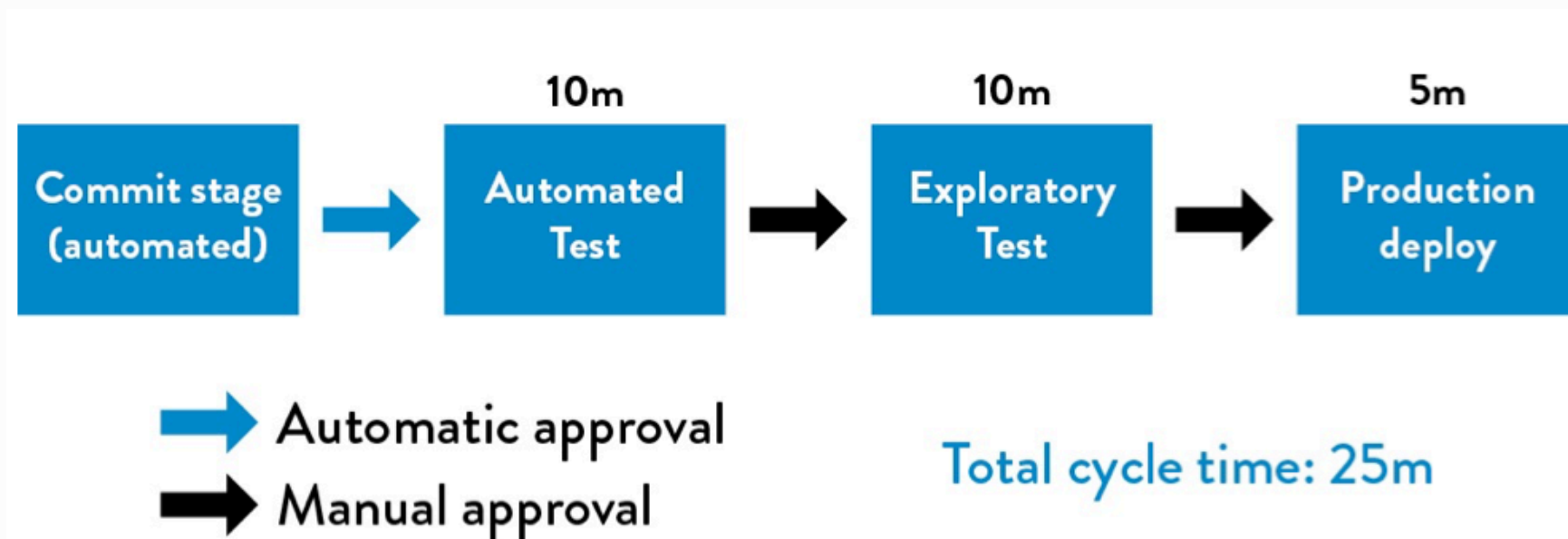


Lead Time: Tempo che intercorre tra la creazione di una richiesta e il completamento di un lavoro.



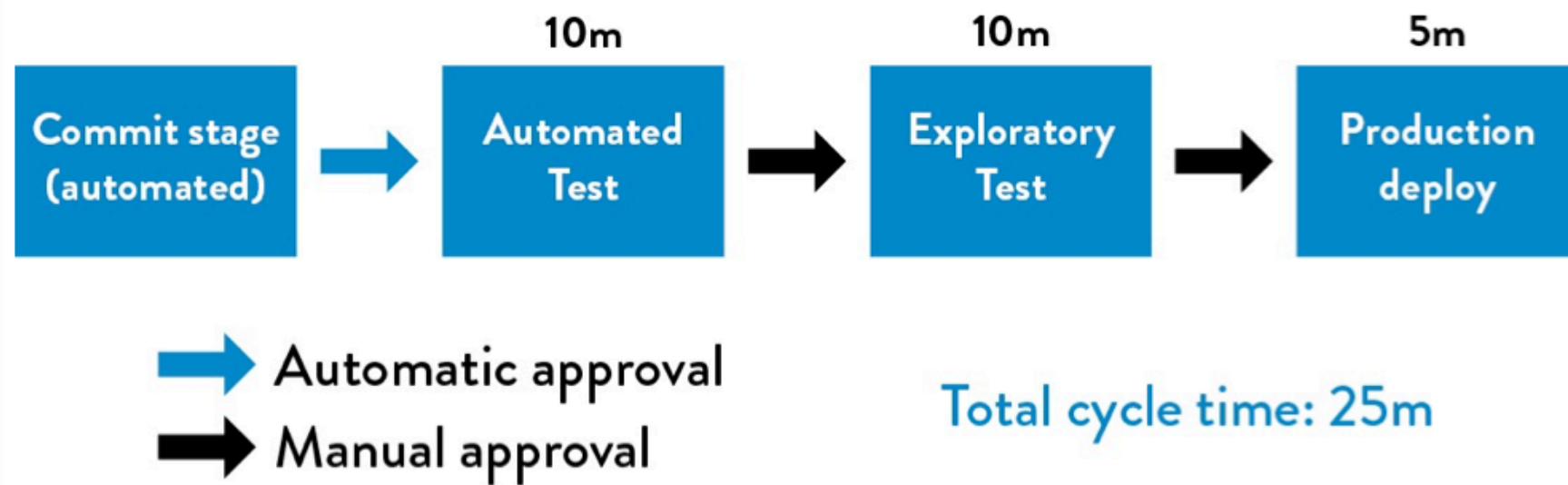
Lead Time: Tempo che intercorre tra la creazione di una richiesta e il completamento di un lavoro.

Scenario Ideale di DevOps: minimizzare parte del lead time in termini di minuti!



Problema!

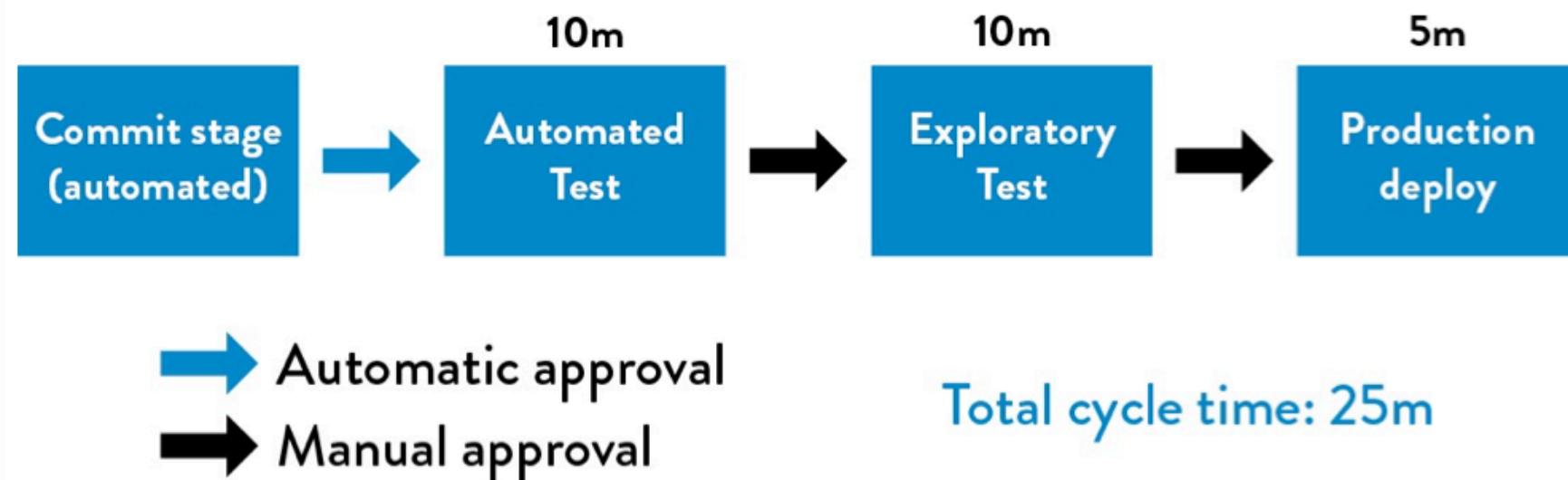
U
S

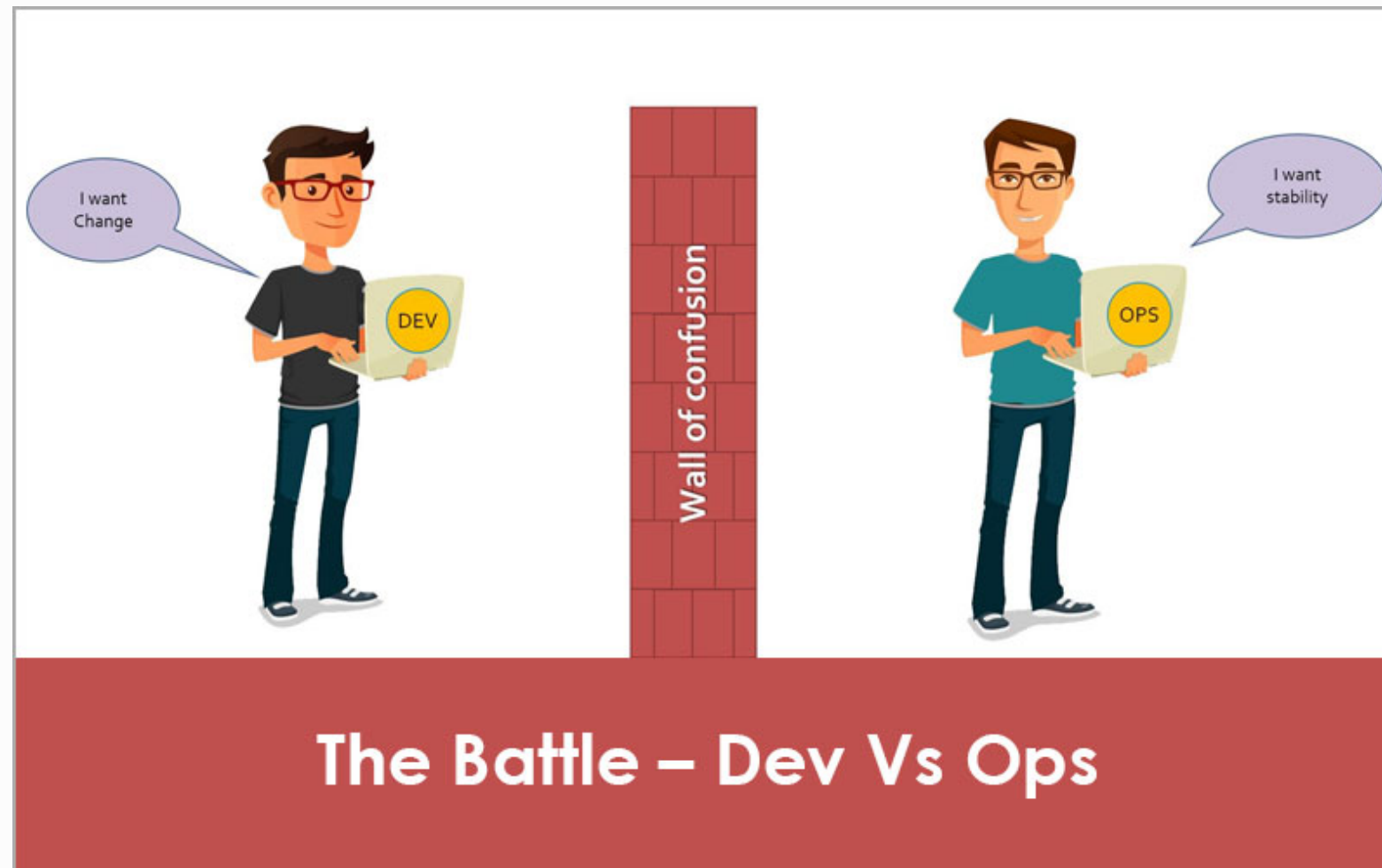


Problema!

Le fasi che vediamo qui sotto non coinvolgono solo il ciclo di vita relativo allo sviluppo del software.

Per poter raggiungere questo obiettivo, DevOps deve interagire sia sulla fase di Development che su quella di Operations!



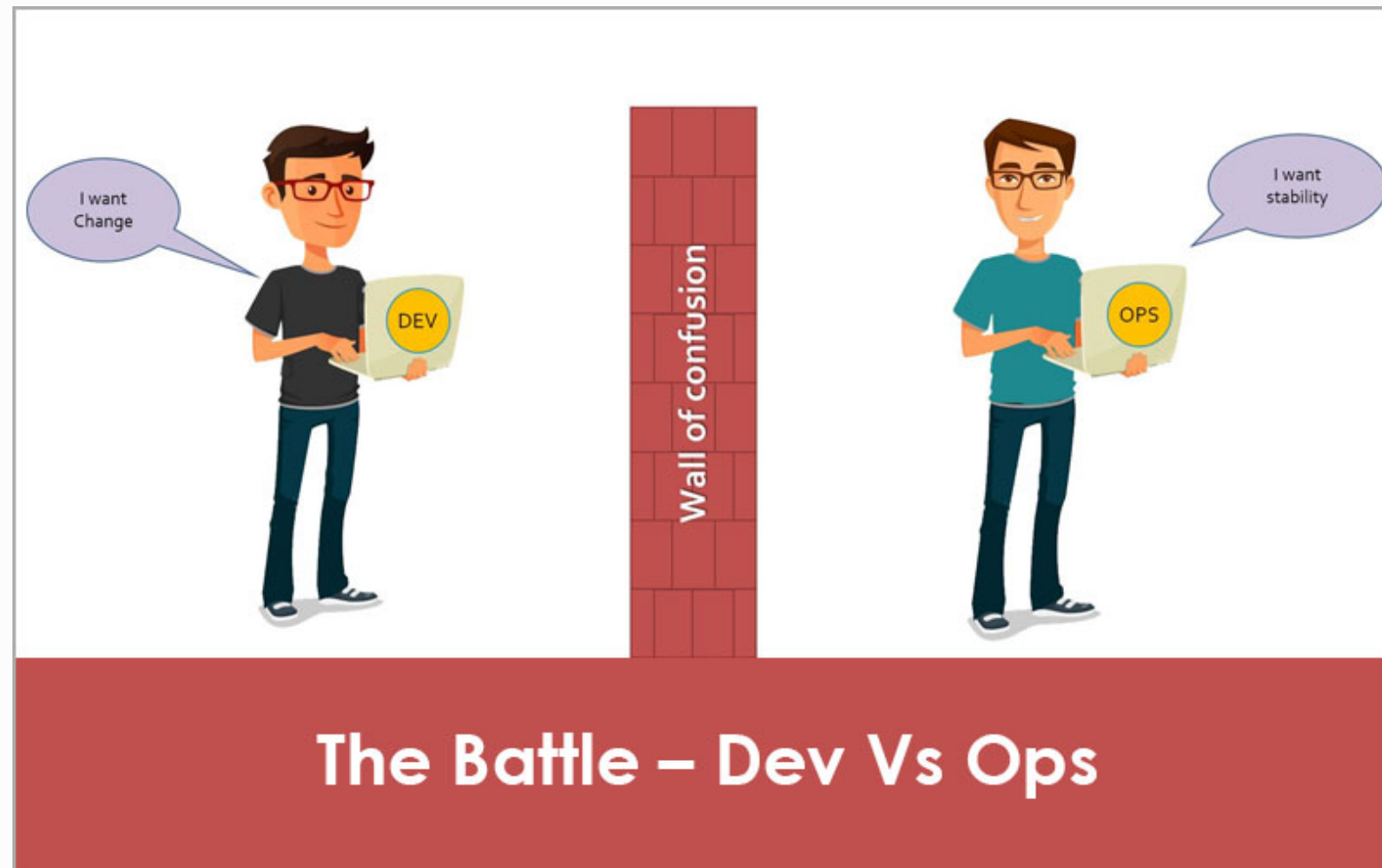


Dev

- Il team Dev punta a fornire **nuove versioni del software**.
- Il team Dev **non** presta attenzione al Quality of Service.

Ops

- Il team Ops punta a **ritardare la messa in produzione** di nuove versioni del software.
- Il team Ops **punta** a garantire il Quality of Service.



DevOps fornisce pratiche e strumenti che colmano il divario tra Dev ed Ops e creare una realtà collaborativa tra i due team.

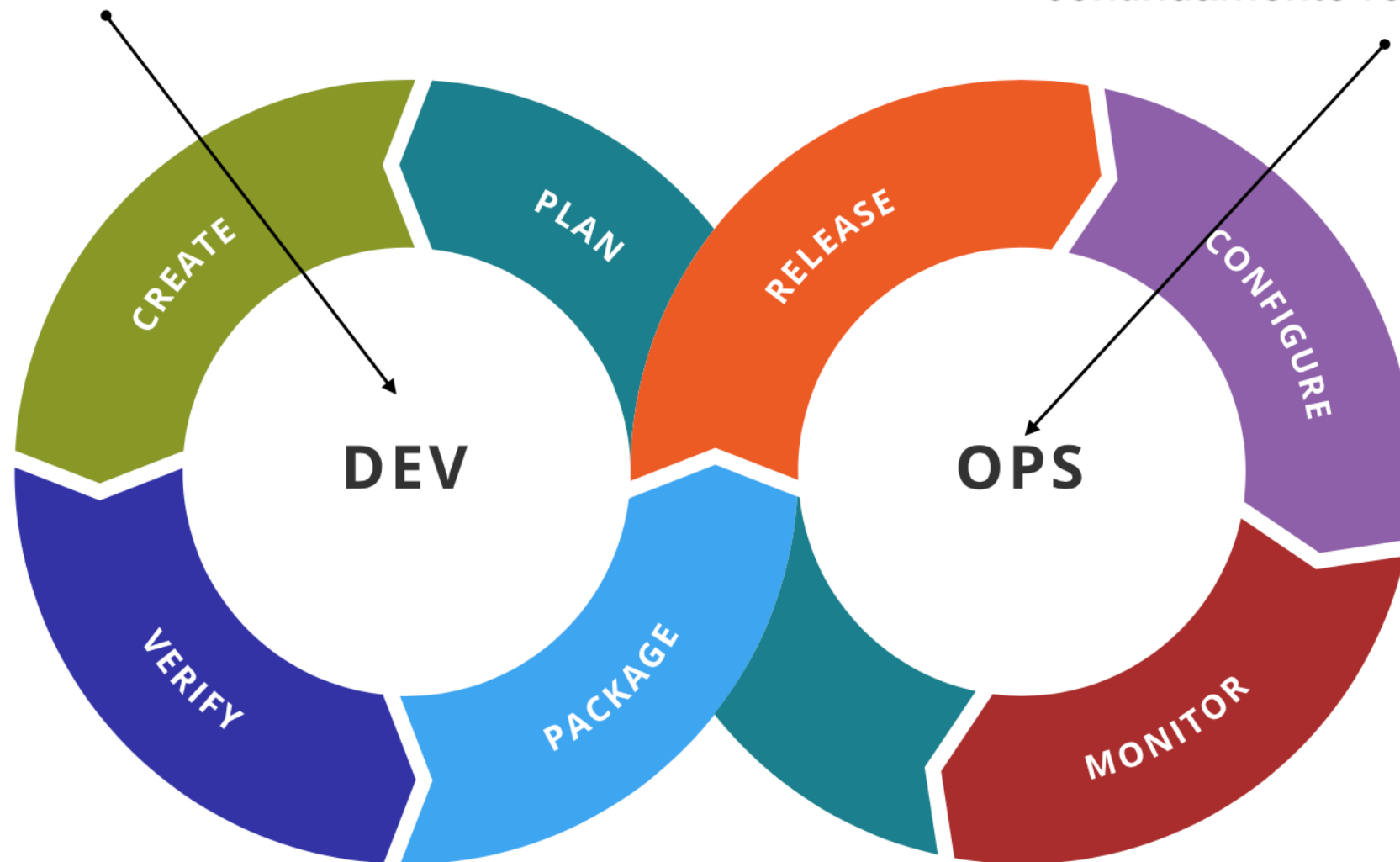
Perché DevOps

Obiettivo di Dev:

"Preparare un'architettura software progettata per essere immediatamente implementabile"

Obiettivo di Ops:

"Osservare il runtime dell'architettura e fornire un feedback operativo allo sviluppatore per migliorare continuamente l'architettura"



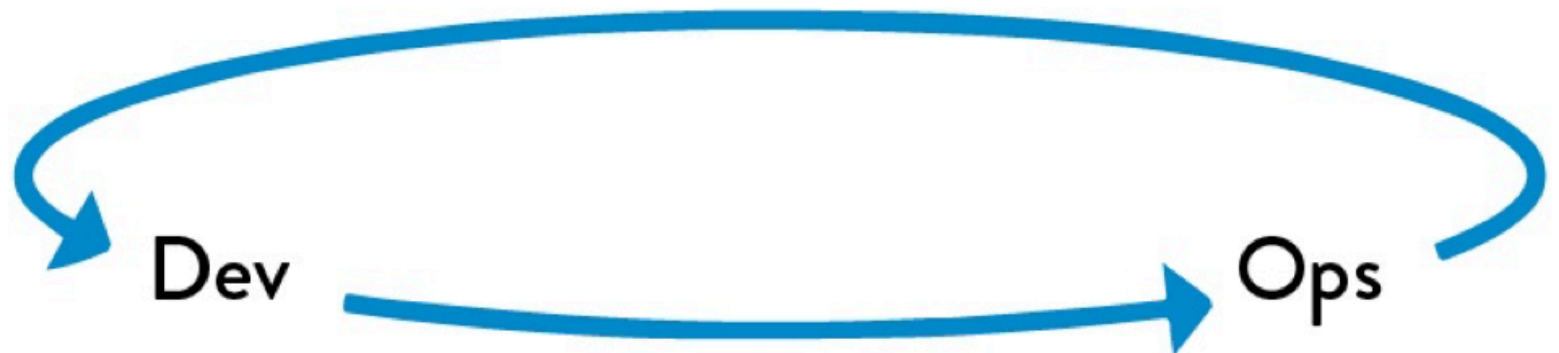
The Three Ways of DevOps

***The Phoenix Project*[1]** definisce I principi su cui si basano tutte le pratiche e gli strumenti che ci permettono di applicare DevOps.

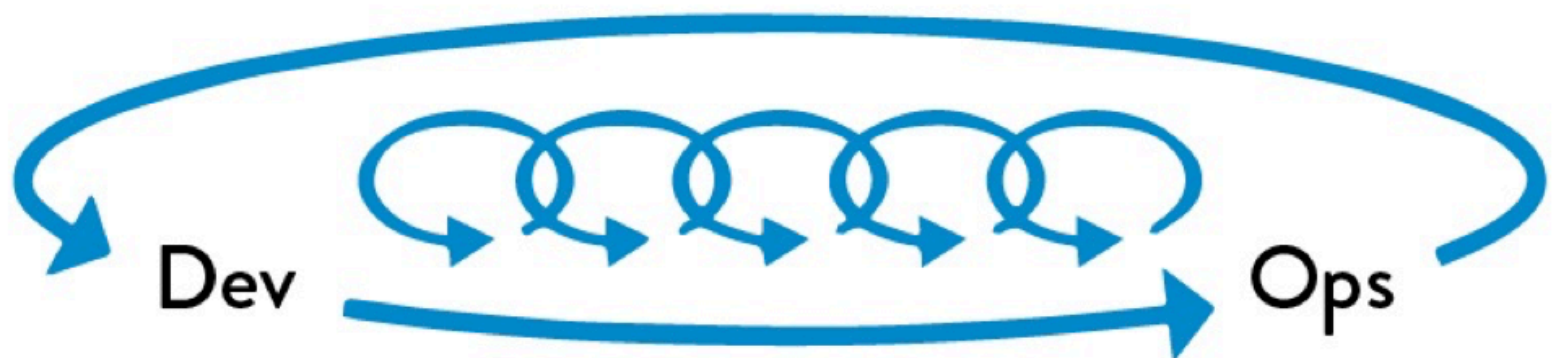
1: Fast left-to-right flow of work



2: Fast and constant flow of feedback



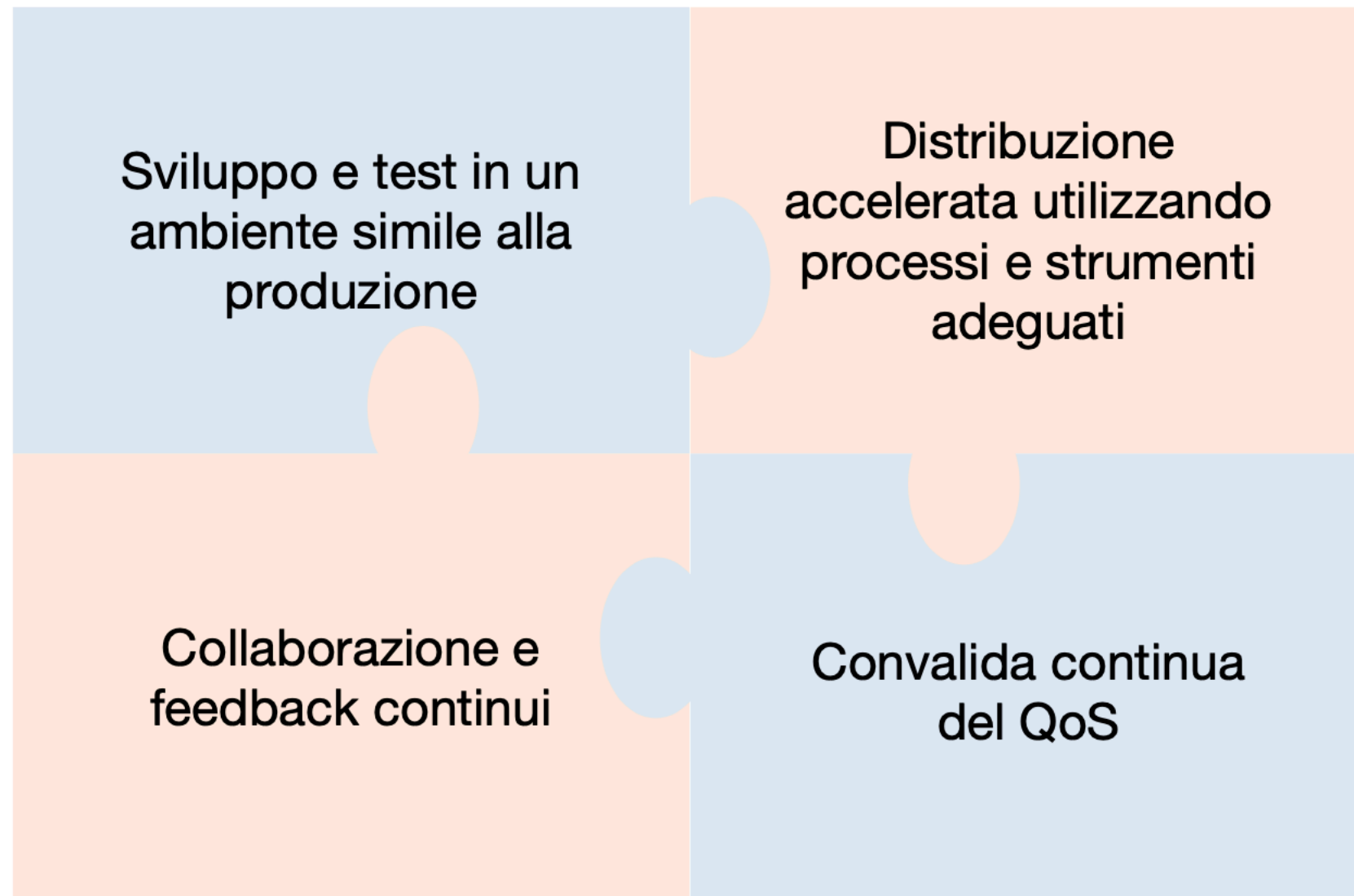
3: Continual Learning and Experimentation



[1] Gene Kim, Kevin Behr, and George Spafford. 2013. *The Phoenix Project: A Novel about IT, DevOps, and Helping Your Business Win* (1st. ed.). IT Revolution Press.

The Three Ways of DevOps

Per garantire i principi di DevOps abbiamo bisogno quindi di:



The Three Ways of DevOps

Per garantire i principi di DevOps abbiamo bisogno quindi di:

Continuous Architecting

Def. “architettare per testare, costruire e distribuire, prendendo in considerazione gli attributi di qualità e sfruttando il feedback dal runtime”

Continuous Integration

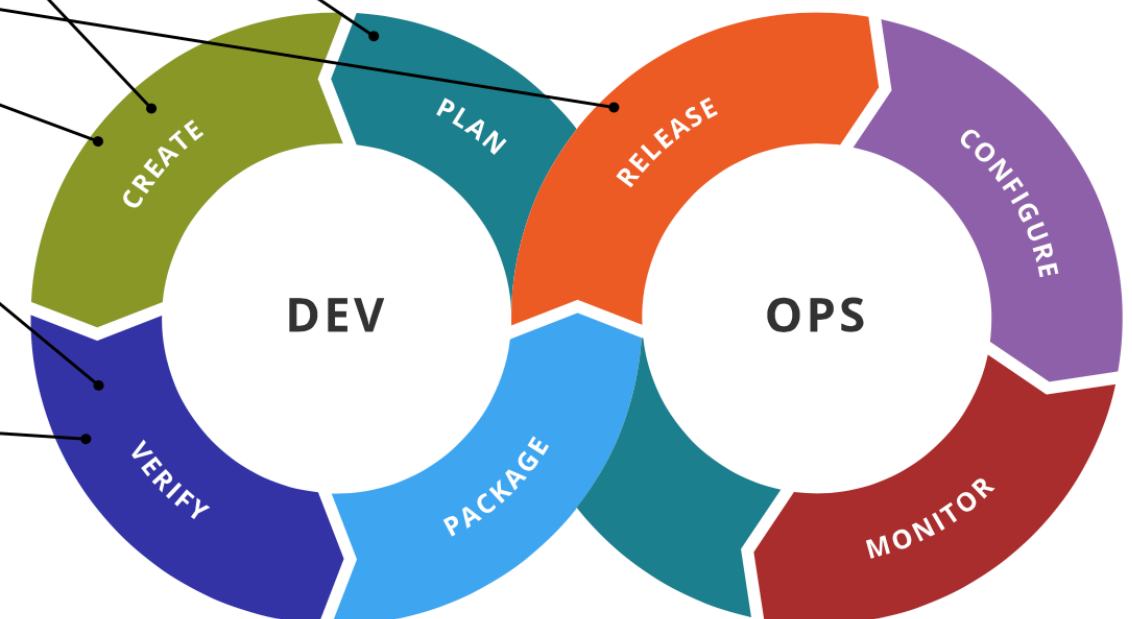
Def. “unire frequentemente le copie di lavoro di tutti gli sviluppatori in una linea principale condivisa”

Esempi. Apache Jenkins, Travis-CI.

Continuous Testing

Def. “eseguire test come parte della pipeline di compilazione in modo che ogni distribuzione sia convalidata”

Esempi. Selenium+JUnit.



Continuous Architecting

L'obiettivo della continuous architecting è quello di garantire che l'architettura del sistema si evolva e si adatti continuamente per soddisfare le esigenze aziendali e tecnologiche in un ambiente di sviluppo e operativo in continua evoluzione.

Ciò include la creazione di un'architettura **modulare, flessibile e scalabile** che consenta la **distribuzione continua del software** e delle **modifiche** nel contesto DevOps.

Le funzionalità chiave della continuous architecting includono:

1. **Microservices: Creazione di architetture basate su Microservizi.**
2. **Containerization:** Creazione di **container isolati** per provare i cambiamenti **della parte di Dev** in ambienti Ops simulati che siano il **più simili possibile alla produzione**.
3. **IaC:** Definizione dell'**infrastruttura del sistema come codice (Infrastructure as Code)**, consentendo una gestione più efficiente, riproducibile e scalabile delle risorse di infrastruttura.
4. **Configuration Management Tools:** Configurazione gestione automatica delle componenti del sistema (o Orchestrazione), garantendo che l'architettura sia coerente e correttamente configurata in tutti gli ambienti.

Continuous Integration

La continuous integration (CI) è una pratica di sviluppo del software che coinvolge **l'integrazione regolare e automatica del codice di sviluppo in un repository condiviso**.

L'obiettivo principale della CI è quello di **ridurre i problemi di integrazione, identificare errori precocemente** come parte di un processo di sviluppo collaborativo e continuo.

Le funzionalità chiave della continuous integration includono:

1. **Automazione delle build:** La CI consente di **automatizzare il processo di creazione dell'eseguibile**, dei pacchetti o del deploy del software. Ciò assicura che il codice sorgente venga compilato in modo coerente e senza errori.
2. **Analisi statica del codice:** La CI può eseguire automaticamente analisi statica del codice per identificare **potenziali problemi, come violazioni di stile, errori comuni o vulnerabilità di sicurezza**.
3. **Notifiche e feedback tempestivi:** La CI **fornisce notifiche immediate** agli sviluppatori sui risultati delle build e dei test. In caso di errori o fallimenti, vengono inviate notifiche per consentire agli sviluppatori di risolvere rapidamente i problemi.
4. **Reportistica e monitoraggio:** La CI **genera report sullo stato delle build**, dei test e delle metriche di qualità del codice, che possono essere consultati per valutare la salute del progetto e l'avanzamento del lavoro.
5. **Versionamento del codice:** La CI tiene traccia delle **diverse versioni del codice sorgente**, consentendo di **ripristinare versioni precedenti** in caso di necessità.

Continuous Testing

Il continuous testing (test continuo) è una pratica che si integra nella Continuous Integration (CI) nella pipeline di sviluppo e delivery del software all'interno di un approccio DevOps.

Consiste nell'**esecuzione automatica e continua dei test** durante tutto il ciclo di vita del software, in modo da identificare tempestivamente eventuali problemi o regressioni e garantire che il software funzioni come previsto.

Le funzionalità principali del continuous testing includono:

1. **Automazione dei test:** Il continuous testing prevede l'automazione dei test, compresi i test unitari, i test di integrazione, i test funzionali, i test di regressione e altri tipi di test. Ciò permette di eseguire i test in modo rapido, ripetibile e affidabile.
2. **Integrazione con la pipeline di CI/CD:** Il continuous testing viene integrato nella pipeline di continuous integration e continuous delivery. Ciò significa che i test vengono eseguiti automaticamente ogni volta che viene effettuata una nuova build o viene apportata una modifica al codice.
3. **Esecuzione dei test in ambienti simili alla produzione:** I test vengono eseguiti in ambienti che sono il più possibile simili a quello di produzione, garantendo una maggiore affidabilità dei risultati dei test e riducendo i rischi di problemi che potrebbero verificarsi solo in ambienti diversi.
4. **Test paralleli e distribuiti:** Il continuous testing può essere **eseguito in parallelo** su più ambienti o su diverse istanze di test per accelerare i tempi di esecuzione e ottenere risultati più rapidi.

The Three Ways of DevOps

Per garantire i principi di DevOps abbiamo bisogno quindi di:

PERIODIC TABLE OF DEVOPS TOOLS (V2)																																			
Os Open Source Fr Free Fm Freemium Pd Paid En Enterprise SCM CI Deployment Cloud / IaaS / PaaS BI / Monitoring Database Mgmt Repo Mgmt Config / Provisioning Release Mgmt Logging Build Testing Containerization Collaboration Security																																			

XebiaLabs

Follow @xebialabs

91 En Xlr XL Release	92 En Ur UrbanCode Release	93 En Bm BMC Release Process	94 En Hp HP Codar	95 En Au Automic	96 En Pl Plutora Release	97 En Sr Serena Release	98 Pd Tfs Team Foundation	99 Fm Tr Trello	100 Pd Jr Jira	101 Fm Rf HipChat	102 Fm Sl Slack	103 Fm Fd Flowdock	104 Pd Pv Pivotal Tracker	105 En Sn ServiceNow
106 Os Ki Kibana	107 Fm Nr New Relic	108 En Dt Dynatrace	109 Os Ni Nagios	110 Os Zb Zabbix	111 En Dd Datadog	112 Os El Elasticsearch	113 Fm Ad AppDynamics	114 En Sp Splunk	115 Fm Le Logentries	116 Fm Sl Sumo Logic	117 Os Ls Logstash	118 Os Sn Snort	119 Os Tr Tripwire	120 En Ff Fortify

Ingegneria, Gestione ed Evoluzione del Software

DevOps

